Lecture_07_Three_Dimensional_Transformations

Lecture 7 — Three-Dimensional Transformations

Lecture focus: Extending transformation ideas from 2D to 3D. This lecture explains translation, scaling, rotation, homogeneous coordinates, 4×4 matrices, composite transformations, coordinate spaces, and instance transformations in 3D graphics.

Learning outcomes

1. Explain why 3D transformations are needed in computer graphics.
2. Represent 3D points and vectors using coordinates.
3. Apply 3D translation, scaling, and rotation.
4. Write standard 3D rotation matrices around the x, y, and z axes.
5. Explain homogeneous coordinates in 3D.
6. Use 4×4 transformation matrices.
7. Explain composite transformations and why order matters.
8. Describe local, world, view, and screen coordinate spaces.
9. Explain instance transformations in a 3D scene.
10. Relate 3D transformations to games, CAD, robotics, AR/VR, animation, and GPU rendering.

---

1. From 2D transformations to 3D transformations

In Lecture 4, we studied transformations in 2D. We used translation, rotation, scaling, shear, reflection, homogeneous coordinates, and matrix composition.

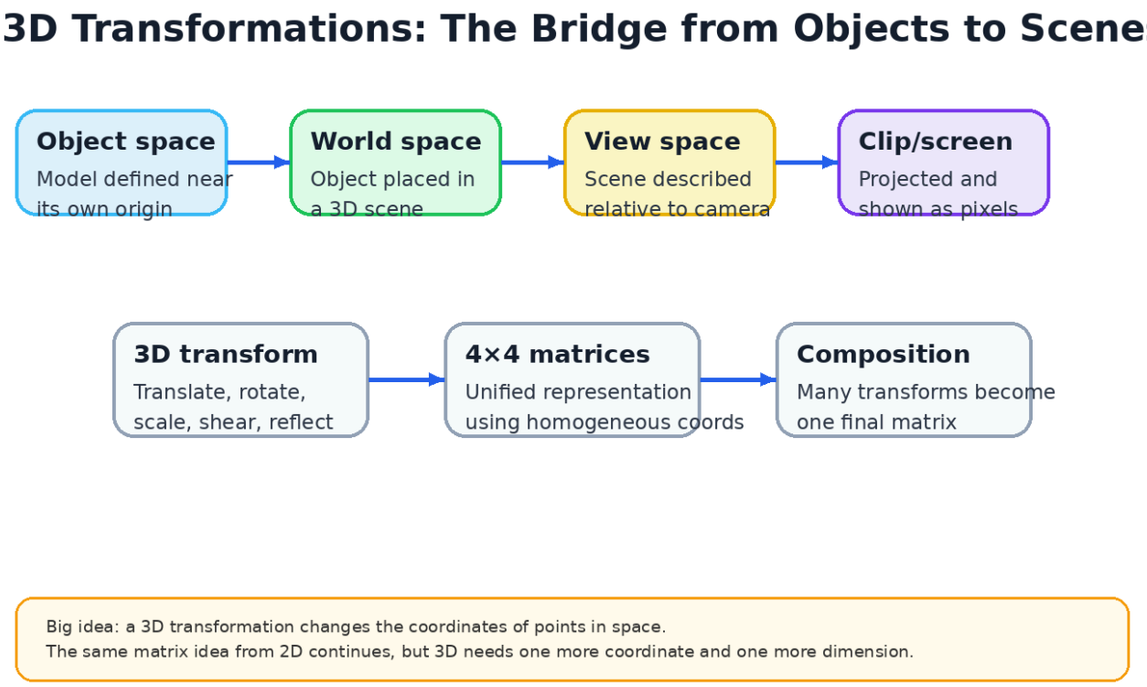
In 3D, the same general idea continues:

```
old point → transformation → new point
```

But now each point has three coordinates:

```
P = (x, y, z)
```

and transformations must handle depth as well as horizontal and vertical movement.



3D transformations are essential because objects in a scene are rarely created exactly where they should appear. A model is usually created in its own convenient coordinate system, then moved, rotated, and scaled into the final scene.

Simple mental model

A 3D transformation answers questions like:

- Where is the object?
- How large is it?
- Which direction is it facing?
- How does it move over time?
- How is it positioned relative to the camera?

---

2. 3D coordinate system

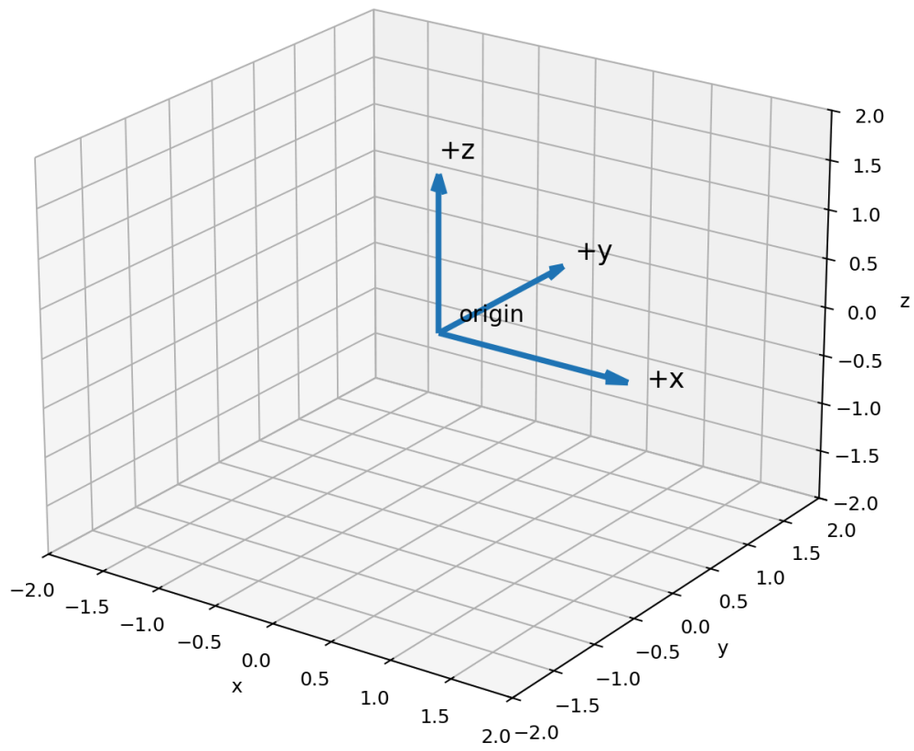
A 3D coordinate system has three axes:

```
x-axis
y-axis
z-axis
```

A point in 3D is written as:

```
P = (x, y, z)
```

Right-handed 3D coordinate system



Right-handed coordinate system

Many graphics and mathematics texts use a **right-handed coordinate system**.

A common way to remember it:

- point the fingers of your right hand from +x toward +y,
- your thumb points toward +z.

“figures/right-handed-coordinate-system.jpg” could not be found.

However, not all graphics systems use the same convention. Some engines use left-handed coordinate systems.

Important warning

The sign of rotation and the meaning of depth may differ between graphics systems. Always check the coordinate convention used by the API, library, or textbook.

---

3. Points vs vectors in 3D

A **point** represents a position:

```
P = (x, y, z)
```

A **vector** represents a direction and magnitude:

```
v = (vx, vy, vz)
```

This distinction matters because translation affects points but not pure direction vectors.

Example:

```
Point:  position of a vertex
Vector: direction of light, surface normal, velocity
```

In homogeneous coordinates, we often represent them differently:

```
Point:  [x, y, z, 1]
Vector: [x, y, z, 0]
```

The last coordinate helps translation apply to points but not to direction vectors.

---

4. Translation in 3D

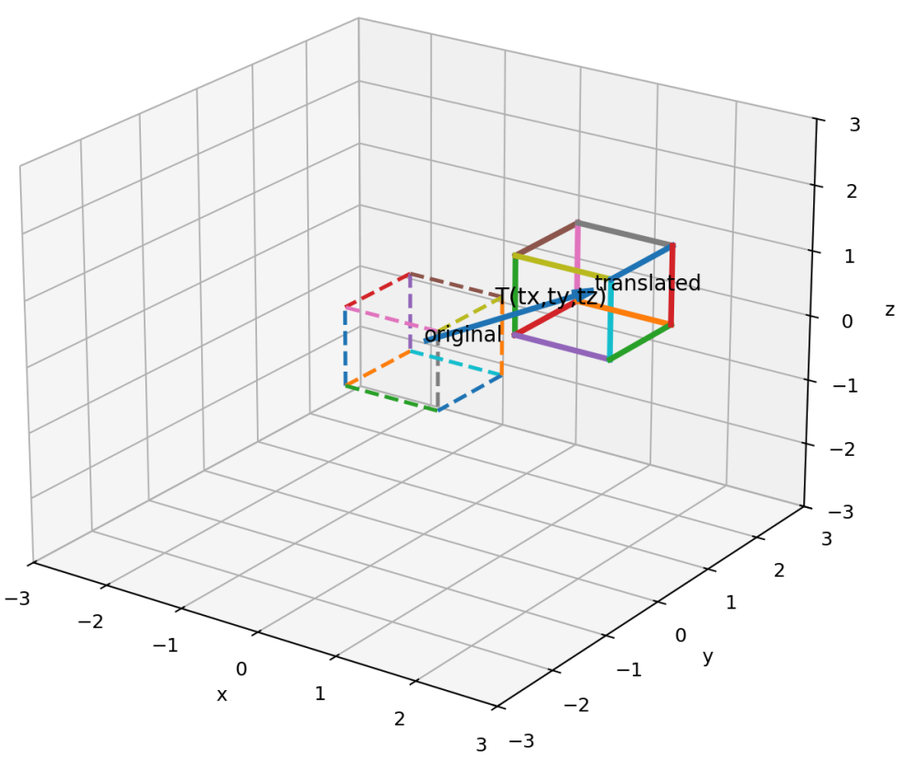
3D translation moves every point by a fixed amount:

```
tx in x-direction
ty in y-direction
tz in z-direction
```

The equations are:

```
x′ = x + tx
y′ = y + ty
z′ = z + tz
```

Translation in 3D: P′ = P + T



Example

Translate point:

```
P = (2, 3, 4)
```

by:

```
T = (5, −1, 2)
```

Then:

```
x′ = 2 + 5  = 7
y′ = 3 − 1  = 2
z′ = 4 + 2  = 6
```

So:

```
P′ = (7, 2, 6)
```

Translation matrix

Using homogeneous coordinates:

```
[ x′ ]   [ 1  0  0  tx ] [ x ]
[ y′ ] = [ 0  1  0  ty ] [ y ]
[ z′ ]   [ 0  0  1  tz ] [ z ]
[ 1  ]   [ 0  0  0   1 ] [ 1 ]
```

---

5. Scaling in 3D

3D scaling changes the size of an object along the x, y, and z axes.

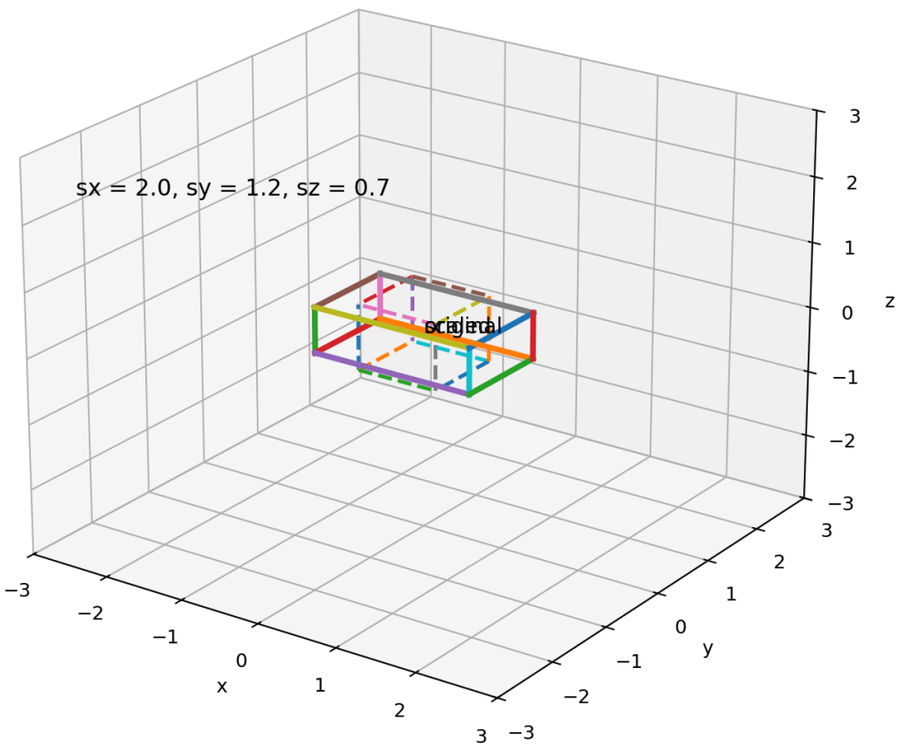
The scale factors are:

```
sx, sy, sz
```

The equations are:

```
x′ = sx x
y′ = sy y
z′ = sz z
```

Scaling in 3D



Uniform scaling

If:

```
sx = sy = sz
```

then the object grows or shrinks equally in all directions.

The shape remains similar.

Non-uniform scaling

If the scale factors are different:

```
sx ≠ sy ≠ sz
```

then the object is stretched or compressed differently along different axes.

Scaling matrix

```
[ x′ ]   [ sx  0   0   0 ] [ x ]
[ y′ ] = [ 0   sy  0   0 ] [ y ]
[ z′ ]   [ 0   0   sz  0 ] [ z ]
[ 1  ]   [ 0   0   0   1 ] [ 1 ]
```

Application

Scaling is used to resize 3D models. A single chair model can be scaled to create small chairs, large chairs, or stylized furniture. In games, scaling can also be used for squash-and-stretch cartoon effects.

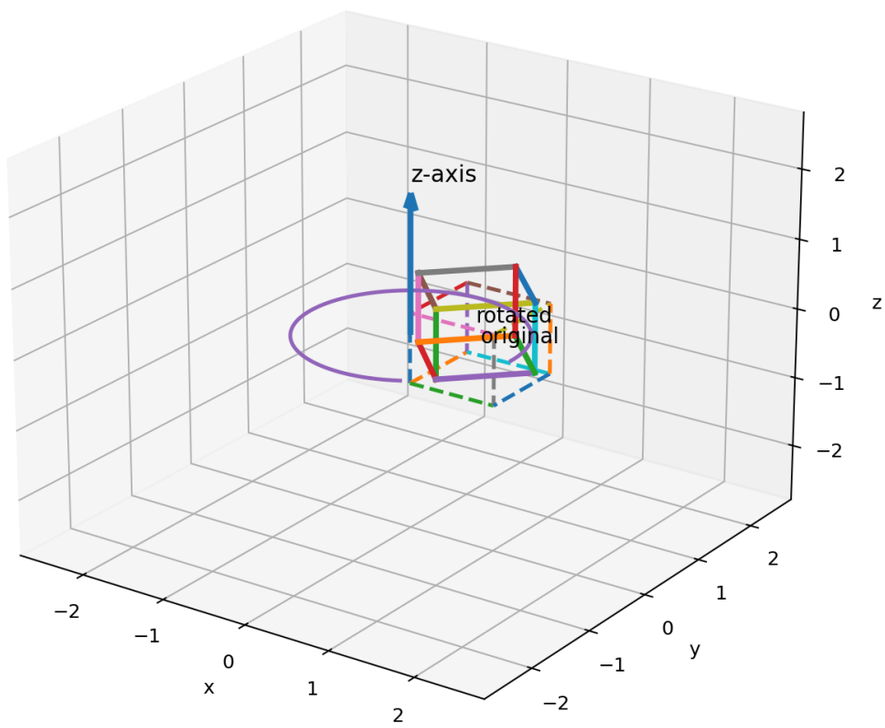
6. Rotation in 3D

Rotation in 2D is simple because there is only one plane of rotation.

In 3D, rotation is more complex because an object can rotate around different axes:

```
x-axis
y-axis
z-axis
```

Rotation around the z-axis



A rotation around the z-axis is similar to the familiar 2D rotation in the xy-plane.

---

7. Rotation around the x-axis

Rotation around the x-axis leaves x unchanged and rotates the y-z coordinates.

```
x′ = x
y′ = y cosθ − z sinθ
z′ = y sinθ + z cosθ
```

Matrix:

```
[ 1    0       0   ]
[ 0   cosθ   −sinθ ]
[ 0   sinθ    cosθ ]
```

Using homogeneous coordinates:

```
[ 1    0       0    0 ]
[ 0   cosθ   −sinθ  0 ]
[ 0   sinθ    cosθ  0 ]
[ 0    0       0    1 ]
```

---

8. Rotation around the y-axis

Rotation around the y-axis leaves y unchanged and rotates the x-z coordinates.

```
x′ = x cosθ + z sinθ
y′ = y
z′ = −x sinθ + z cosθ
```

Matrix:

```
[  cosθ   0   sinθ ]
[   0     1    0   ]
[ −sinθ   0   cosθ ]
```

Using homogeneous coordinates:

```
[  cosθ   0   sinθ   0 ]
[   0     1    0     0 ]
[ −sinθ   0   cosθ   0 ]
[   0     0    0     1 ]
```

---

9. Rotation around the z-axis

Rotation around the z-axis leaves z unchanged and rotates the x-y coordinates.

```
x′ = x cosθ − y sinθ
y′ = x sinθ + y cosθ
z′ = z
```

Matrix:

```
[ cosθ  −sinθ   0 ]
[ sinθ   cosθ   0 ]
[  0      0     1 ]
```

Using homogeneous coordinates:

```
[ cosθ  −sinθ   0   0 ]
[ sinθ   cosθ   0   0 ]
[  0      0     1   0 ]
[  0      0     0   1 ]
```

Basic 3D Rotation Matrices

Rotation about x-axis

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta \\ 0 & \sin\theta & \cos\theta \end{bmatrix}$$

Rotation about y-axis

$$\begin{bmatrix} \cos\theta & 0 & \sin\theta \\ 0 & 1 & 0 \\ -\sin\theta & 0 & \cos\theta \end{bmatrix}$$

Rotation about z-axis

$$\begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Mnemonic: rotating around one axis leaves that axis coordinate unchanged.
Rx leaves x unchanged, Ry leaves y unchanged, and Rz leaves z unchanged.
Sign conventions depend on right-handed vs left-handed coordinate systems and row-vector vs colun

Useful mnemonic

The axis of rotation remains unchanged.

- Rotation about x keeps x unchanged.
- Rotation about y keeps y unchanged.
- Rotation about z keeps z unchanged.

---

10. Homogeneous coordinates in 3D

Just as 2D used 3×3 matrices, 3D commonly uses **4×4 matrices**.

A 3D point:

```
(x, y, z)
```

is represented as:

```
[x, y, z, 1]
```

“figures/fig07_homogeneous_3d_matrix.png” could not be found.

A general 3D affine transformation can be represented as:

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} a & b & c & t_x \\ d & e & f & t_y \\ g & h & i & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

The upper-left 3×3 part usually contains rotation, scaling, shear, or reflection.

The last column contains translation.

Why 4×4 matrices are standard

They allow a graphics system to combine:

- translation,
- rotation,
- scaling,
- shear,
- reflection,
- camera transformations,
- projection transformations,

into a unified matrix framework.

---

11. Composite transformations in 3D

A real object usually needs more than one transformation.

Example:

```
scale → rotate → translate
```

Using matrices, we can combine them:

```
M = T × R × S
```

Then transform a point:

```
P′ = M × P
```

With column vectors, the rightmost matrix acts first.

So:

```
P′ = T × R × S × P
```

means:

1. scale first,
2. rotate second,
3. translate third.

---

12. Transformation order matters

Matrix multiplication is not generally commutative:

```
T × R ≠ R × T
```

So changing the order changes the final result.

“figures/fig08_order_matters_3d.png” could not be found.

Intuitive explanation

If you rotate an object around the origin and then translate it, it ends up in one place.

If you translate it away from the origin and then rotate it, the translation vector itself is affected by the rotation. The object may swing around the origin like it is attached to a long arm.

This is why order is one of the most important ideas in transformation problems.

---

13. Rotation about an arbitrary axis

So far, we considered rotation around the x, y, and z axes.

But in real graphics, we often need to rotate around an arbitrary axis.

Examples:

- rotating a door around a hinge line,
- rotating a wheel around its axle,
- rotating a camera around a target,
- rotating a bone around a skeleton joint axis.

“figures/fig10_arbitrary_axis_rotation.png” could not be found.

There are several ways to handle arbitrary axis rotation:

1. translate the axis to pass through the origin,
2. align the arbitrary axis with a coordinate axis,
3. rotate around that coordinate axis,
4. undo the alignment,
5. translate back.

A more advanced mathematical method is **Rodrigues' rotation formula**.

In game engines, rotations are also commonly represented using **quaternions**, especially for smooth animation and avoiding some rotation problems.

Trivia: gimbal lock

Euler angles

Euler angles are a set of 3 sequential angles (α,β,γ
) that describe 3D orientations, and follow Euler's rotation theorem:

> Euler’s rotation theorem: Any arbitrary orientation in three-dimensional space can be described with only three angles.

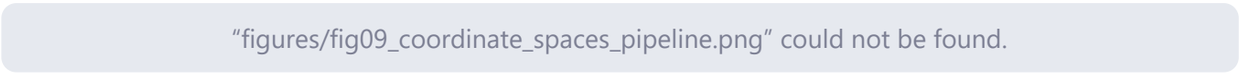
More about Euler angles

When rotations are represented using Euler angles, certain orientations can cause two rotation axes to align. This is called **gimbal lock**. Quaternions are often used in animation and game engines to avoid this issue.

“figures/Gimbal_Lock_Plane.gif” could not be found.

14. Coordinate spaces in a 3D pipeline

A 3D object usually travels through several coordinate spaces before becoming pixels.



Local or model space

The object is defined in its own coordinate system.

Example:

```
A car model may be built around its own center.
```

World space

The object is placed into the scene.

Example:

```
The car is placed on a road at position (20, 0, 5).
```

View or camera space

The world is described relative to the camera.

Example:

```
Objects in front of the camera are prepared for projection.
```

Clip and screen space

After projection and clipping, the final visible result is mapped to screen pixels.

This pipeline becomes central in later lectures on projection and 3D viewing.

---

15. Model matrix, view matrix, and projection matrix

A very common graphics pipeline uses three major matrices:

```
Model matrix
View matrix
Projection matrix
```

Model matrix

The model matrix transforms points from local space to world space.

```
local → world
```

It places the object in the scene.

View matrix

The view matrix transforms points from world space to camera space.

```
world → view
```

It represents the camera's position and orientation.

Projection matrix

The projection matrix transforms 3D camera coordinates toward a 2D image.

```
view → clip/projection space
```

Projection will be discussed in a later lecture.

Combined form

Many rendering systems combine these:

```
MVP = Projection × View × Model
```

Then:

```
clip_position = MVP × local_position
```

This is one of the most common formulas in modern graphics programming.

---

16. Instance transformations

Instance transformation means reusing one model multiple times with different transformation matrices.

Example:

```
Tree model
    instance 1: translate(5, 0, 10), scale(1.0)
    instance 2: translate(9, 0, 12), scale(0.7), rotate(20°)
    instance 3: translate(12, 0, 8), scale(1.4), rotate(−10°)
```

Instead of storing three separate tree models, the system stores one model and uses different matrices.

Why this matters

Instance transformations save:

- memory,
- modeling effort,
- rendering preparation,
- design time.

They are used in:

- forests,
- crowds,
- repeated buildings,
- particles,
- CAD blocks,
- game props,
- UI objects in 3D interfaces.

---

17. Normals and why they need special care

A **normal vector** is a vector perpendicular to a surface. Normals are extremely important for lighting.

“figures/fig11_normal_transformation.png” could not be found.

For simple rotation, transforming normals is straightforward.

But for non-uniform scaling, normals cannot always be transformed exactly like ordinary points.

In many graphics pipelines, normals are transformed using:

```
inverse-transpose of the linear part of the model matrix
```

This is often called the **normal matrix**.

Why mention this now?

You do not need to derive this fully at this stage. But it is useful to know because lighting errors often happen when normals are transformed incorrectly.

---

18. Applications of 3D transformations

“figures/fig12_3d_transform_applications.png” could not be found.

Games

Characters, vehicles, weapons, cameras, and world objects all use transformations.

Animation

Skeleton joints use chains of transformations. A hand moves because the shoulder, elbow, and wrist transformations combine.

CAD and robotics

Robotic arms use transformations to describe joint positions and tool orientation.

AR and VR

Virtual objects must be positioned relative to the real world and the user's head or camera.

Scientific visualization

3D data, molecules, medical scans, and simulations all require coordinate transformations.

GPU rendering

The rendering pipeline depends heavily on model, view, and projection matrices.

---

19. Worked examples

Example 1: 3D translation

Translate:

```
P = (1, 2, 3)
```

by:

```
T = (4, −2, 5)
```

Then:

```
P′ = (1 + 4, 2 − 2, 3 + 5)
P′ = (5, 0, 8)
```

Example 2: 3D scaling

Scale:

```
P = (2, 3, 4)
```

by:

```
sx = 2, sy = 1, sz = 3
```

Then:

```
x′ = 2 × 2 = 4
y′ = 1 × 3 = 3
z′ = 3 × 4 = 12
```

So:

```
P′ = (4, 3, 12)
```

Example 3: rotation around z-axis by 90°

Rotate:

```
P = (3, 2, 5)
```

around the z-axis by 90°.

For 90°:

```
cosθ = 0
sinθ = 1
```

So:

```
x′ = x cosθ − y sinθ = 3(0) − 2(1) = −2
y′ = x sinθ + y cosθ = 3(1) + 2(0) = 3
z′ = z = 5
```

Answer:

```
P′ = (−2, 3, 5)
```

Example 4: composite transformation

A point:

```
P = (1, 1, 1)
```

is first scaled by 2 in all directions, then translated by `(3, 0, 1)`.

Scaling:

```
(1, 1, 1) → (2, 2, 2)
```

Translation:

```
(2, 2, 2) → (5, 2, 3)
```

Final answer:

```
P′ = (5, 2, 3)
```

If translation came first, the answer would be different.

---

20. Common student mistakes

1. Treating 3D rotation as if there is only one possible rotation plane.
2. Forgetting that rotation around an axis leaves that axis coordinate unchanged.
3. Mixing row-vector and column-vector conventions.
4. Writing matrices in the wrong multiplication order.
5. Confusing local space and world space.

6. Forgetting that translation affects points but not direction vectors.
7. Scaling about the origin when intending to scale about the object center.
8. Ignoring coordinate-system handedness.
9. Using degrees in a function that expects radians.
10. Transforming normals incorrectly under non-uniform scaling.

---

21. Short exam-style questions

1. What is the difference between a 2D point and a 3D point?
2. Write the 4×4 matrix for translation by `(tx, ty, tz)`.
3. Write the 4×4 matrix for scaling by `(sx, sy, sz)`.
4. Which coordinate remains unchanged during rotation around the x-axis?
5. Write the rotation matrix around the z-axis.
6. Why do we use homogeneous coordinates in 3D?
7. Why does transformation order matter?
8. What is the difference between local space and world space?
9. What is an instance transformation?
10. Why do surface normals need special care under non-uniform scaling?

---

22. Practice problems

Problem 1

Translate:

```
P = (2, −1, 5)
```

by:

```
T = (3, 4, −2)
```

Find `P′`.

Problem 2

Scale:

```
P = (3, 2, 1)
```

by:

```
sx = 2, sy = 3, sz = 4
```

Find `P′`.

Problem 3

Rotate:

```
P = (4, 1, 2)
```

around the z-axis by 90° counterclockwise.

Problem 4

A point is first translated by `(1, 0, 0)` and then rotated around the z-axis by 90°. Compute the result for:

```
P = (1, 1, 0)
```

Then reverse the order and compare.

Problem 5

Explain the coordinate-space path:

```
local → world → view → clip → screen
```

in your own words.

---

23. Lecture summary

"figures/fig13_lecture_summary.png" could not be found.

The big takeaways are:

- 3D transformations extend 2D transformation ideas into x, y, and z.
- Translation moves objects in 3D space.
- Scaling changes size along the x, y, and z directions.
- Rotation can occur around the x, y, z axes or an arbitrary axis.
- Homogeneous coordinates allow 3D transformations to be represented using 4×4 matrices.
- Composite transformations combine many operations into one matrix.
- Transformation order matters.
- Objects move through local, world, view, clip, and screen coordinate spaces.
- Instance transformations reuse one model many times.
- 3D transformations are foundational in games, CAD, robotics, animation, AR/VR, and GPU rendering.

---

24. Final trivia box

- Most modern 3D engines store transformations as position, rotation, and scale, but internally they often convert these into matrices.
- Game engines commonly use quaternions for rotation because they help avoid gimbal lock and interpolate smoothly.
- A robot arm and a 3D animated character skeleton are mathematically similar: both are chains of transformations.

- The famous MVP formula — `Projection × View × Model × Vertex` — is one of the most important formulas in real-time graphics.
- Every time a game camera moves, the view matrix changes.